파이썬 완전 백과사전 2권 · 딥러닝 심화: 언어와 생성

4일차

남이 만든 언어 모델을 쓰고, 재고, 고치기

16~20차시 · 책 14장 LLM 활용과 LoRA

한 차시 50분 · 하루 5차시 · 예제: examples/vol2/14-llm-lora

오늘의 시간표

차시	주제	책 범위
16	사전학습 언어 모델, 다음 토큰 예측	14장 들어가며 ~ 14.2
17	디코딩, 프롬프트와 채팅 템플릿	14.3 ~ 14.4
18	작은 모델의 한계, 메모리 계산과 양자화	14.5 ~ 14.6
19	LoRA 아이디어, peft로 LoRA 붙이기	14.7 ~ 14.8
20	1권 도우미 미세조정 실습, 14장 정리	14.9 ~ 14.10

수업 전에 내려받기

Qwen2.5-0.5B-Instruct 약 0.95GB

모델 쓰기 (실측 최대)

메모리 약 3.3GB

LoRA 학습 (실측 최대)

배치 8: 약 7.9GB · 배치 2: 약 4.4GB

한 차시 50분: 도입 5분 · 설명과 시연 30분 · 직접 실행 10분 · 정리 5분 · GPU 없이 CPU만으로 모두 돈다

16차시

16

사전학습 언어 모델과 다음 토큰 예측

책 14장 들어가며 ~ 14.2절

학습 목표

- 사전학습 모델을 이름 하나로 불러와 크기와 구조를 숫자로 읽는다
- 토크나이저가 한국어를 어떻게 자르는지 확인한다
- "다음 토큰 맞히기"를 생성 루프로 직접 짜 본다

14장 휴대폰 자판은 어떻게 다음 단어를 알까

휴대폰 자판

"오늘 저녁에" 다음에
뭐 · 같이 · 시간

마음을 읽은 게 아니라 자주 나온 다음 말을 보여 줄 뿐

대규모 언어 모델

같은 놀이, 규모만 다르다

수조 토큰을 읽고 수억~수천억 개 숫자에 "이 글 다음에 무엇이 올까"를 새긴다

오늘의 모델

Qwen2.5-0.5B-Instruct

0.5B = 파라미터 약 5억 개
Instruct = 답하도록 추가 학습
Apache-2.0, 로그인 없이 받는다

9~10차시에서 본 작은 GPT는 직접 만들어 몇 분 학습한 모델, 오늘은 남이 큰 컴퓨터로 오래 학습해 공개한 **사전학습 모델**을 쓴다

14.1 이름 하나로 모델과 토크나이저 받기

```
이름 = "Qwen/Qwen2.5-0.5B-Instruct"  # Hugging Face에 올라 있는 모델 이름
토크나이저 = AutoTokenizer.from_pretrained(이름)  # 글자 ↔ 번호 변환기
모델 = AutoModelForCausalLM.from_pretrained(이름, dtype=torch.float32)

print(type(모델).__name__)
print(f"파라미터: {sum(p.numel() for p in 모델.parameters()):,}개")
설정 = 모델.config
print("층 수:", 설정.num_hidden_layers, "/ 은닉 크기:", 설정.hidden_size,
      "/ 어텐션 헤드:", 설정.num_attention_heads)
print("어휘 크기:", 설정.vocab_size, "/ 한 번에 읽는 최대 길이:", 설정.max_position_embeddings)
print(isinstance(모델, torch.nn.Module))  # 6장의 nn.Module 그대로
```

```
Qwen2ForCausalLM
파라미터: 494,032,768개
층 수: 24 / 은닉 크기: 896 / 어텐션 헤드: 14
어휘 크기: 151936 / 한 번에 읽는 최대 길이: 32768
True
```

- AutoModelForCausalLM: 앞만 보고 다음을 맞히는 모델
- 처음 한 번만 내려받고 캐시에서 다시 쓴다

16차시 · 14.1 사전학습 언어 모델 불러오기 · 예제 14-llm-lora/b01_load.py (핵심 부분)

14.1 숫자로 읽는 모델의 설계도

494,032,768
파라미터. 기초 과정 FashionMNIST 분류기(235,146개)의 2천 배쯤

24
층 수: 트랜스포머 블록 24개를 쌓았다

896
은닉 크기: 토큰 하나를 숫자 896개로 표현

14
어텐션 헤드 수

151,936
어휘 크기: 다음 토큰을 고를 때마다 이 만큼의 후보

32,768
한 번에 읽을 수 있는 최대 토큰 수

블록 · 은닉 크기 · 헤드는 7~8차시에서 직접 만든 부품 이름 그대로. 크기만 다른 같은 설계도이고, 마지막 줄 True: 이 모델도 nn.Module이다

14.1 글자를 번호로: 토크나이저

```
토크나이저 = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-0.5B-Instruct")

문장 = "파이썬은 재미있다"
번호 = 토크나이저.encode(문장)
print(번호)
print([토크나이저.decode([n]) for n in 번호])  # 번호 하나하나가 어떤 조각인지
print(토크나이저.decode(번호))  # 번호를 다시 글자로

# 같은 뜻의 문장, 토큰은 몇 개일까?
for 글 in ["오늘 날씨가 정말 좋네요.", "The weather is really nice today."]:
    print(f"{len(글):2}글자 → {len(토크나이저.encode(글)):2}토큰: {글}")
```

```
[126793, 12802, 144773, 33704, 129242, 56039, 13165, 19946]
['파', '이', '썬', '은', ' 재', '미', '', '다']
파이썬은 재미있다
14글자 → 10토큰: 오늘 날씨가 정말 좋네요.
33글자 →  7토큰: The weather is really nice today.
```

- 거의 한 글자에 토큰 하나
- "있"은 바이트 조각 둘로 갈라졌다
- 이어서 되돌리면 원래 문장

16차시 · 14.1 글자를 번호로: 토크나이저 · 예제 14-llm-lora/b02_tokens.py (핵심 부분)

14.1 같은 뜻인데 한국어는 토큰이 더 든다

오늘 날씨가 정말 좋네요.

14글자 → 10토큰

The weather is really nice today.

33글자 → 7토큰

왜: 학습 때 영어 글을 훨씬 많이 봐서 영어 조각을 크게 만들어 두었다

그래서: 같은 길이 한도 안에 한국어가 덜 들어가고, 생성도 더 오래 걸린다

16차시 · 14.1 글자를 번호로: 토크나이저 · 예제 14-llm-lora/b02_tokens.py 출력 마지막 두 줄

14.2 모델이 내는 것은 점수표

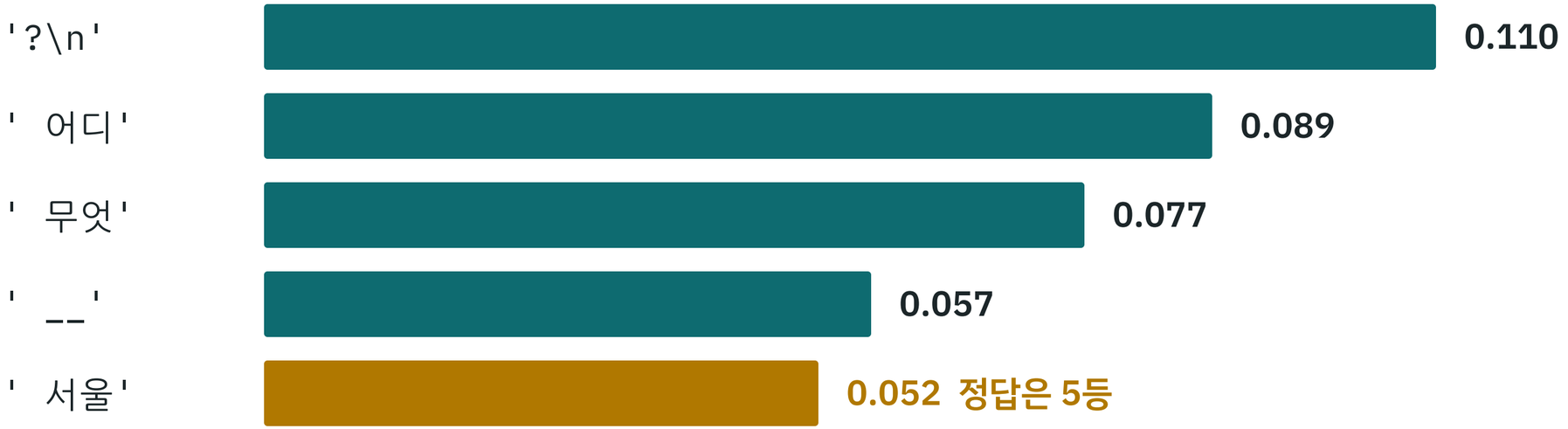
```
입력 = 토크나이저("대한민국의 수도는", return_tensors="pt")
print("입력 번호:", 입력["input_ids"])
with torch.no_grad():
    로짓 = 모델(**입력).logits  # (배치, 토큰 수, 어휘 크기)
print("로짓 모양:", tuple(로짓.shape))
마지막 = 로짓[0, -1]  # 마지막 자리의 점수 = 다음 토큰 후보
확률 = torch.softmax(마지막, dim=-1)
상위 = 확률.topk(5)
for p, n in zip(상위.values, 상위.indices):
    print(f"{p.item():.3f}  {토크나이저.decode([n.item()])!r}")
```

```
입력 번호: tensor([[ 66845,  23573, 125496, 124785,  20401, 134013,  16560]])
로짓 모양: (1, 7, 151936)
```

(1, 7, 151936) = 문장 1개, 토큰 7자리, 자리마다 어휘 151,936개의 점수. 모든 자리에서 동시에 "다음 토큰"의 점수를 내고, 생성할 때는 **마지막 자리**만 쓴다

16차시 · 14.2 모델이 내는 것은 점수표 · 예제 14-llm-lora/b03_next.py (핵심 부분, 상위 5개 출력은 다음 장)

14.2 "대한민국의 수도는" 다음 토큰 상위 5개



모델은 이 글을 **퀴즈 질문의 앞부분**으로 읽었다. 사실을 답하는 게 아니라 "이런 글 다음에 보통 무엇이 오는가"를 답한다

16차시 · 14.2 모델이 내는 것은 점수표 · 예제 14-llm-lora/b03_next.py 출력 다섯 줄을 막대로 (막대 길이 = 확률)

14.2 생성 루프를 직접 짜 보기

```
번호 = 토크나이저("대한민국의 수도는", return_tensors="pt")["input_ids"]
시작_길이 = 번호.shape[1]
with torch.no_grad():
    for _ in range(15):
        다음 = 모델(번호).logits[0, -1].argmax()  # 탐욕(greedy): 1등만 고른다
        번호 = torch.cat([번호, 다음.view(1, 1)], dim=1)
직접 = 토크나이저.decode(번호[0, 시작_길이:])
print(repr(직접))
# 같은 일을 generate 한 줄로 (반복 억제 기본값 1.1을 꺼서 직접 만든 루프와 똑같은 규칙으로)
입력 = 토크나이저("대한민국의 수도는", return_tensors="pt")
with torch.no_grad():
    결과 = 모델.generate(**입력, max_new_tokens=15, do_sample=False, repetition_penalty=1.0)
print("같은가:", 직접 == 자동)
```

```
'?\nA. 서울\nB. 인천\nC. 대구\n'
'?\nA. 서울\nB. 인천\nC. 대구\n'
같은가: True
```

- 1등 '?\n'을 고른 뒤 정말로 객관식 퀴즈를 만들었다
- generate 한 줄과 글자 하나까지 같다

14.2 같은 결과, 다른 비용: KV 캐시

직접 짠 루프

토큰 하나를 붙일 때마다 문장 전체를 처음부터 다시 계산한다

generate

앞 토큰들의 어텐션 계산 결과(키와 값)를 저장해 두고 새 토큰 하나만 계산한다

이 저장소가 **KV 캐시**. 빠른 대신 메모리를 쓴다. 얼마나 쓰는지는 18차시에서 계산한다

직접 실행 10분

1. b03_next.py의 문장을 "서울은 대한민국의"로 바꿔 다음 토큰 상위 5개를 본다. 이번에는 1등이 무엇이고 확률은 얼마인가?
2. b02_tokens.py에 "인공지능은 데이터로 배웁니다."와 "Artificial intelligence learns from data."를 넣어 글자 수와 토큰 수를 비교한다
3. b04_loop.py의 range(15)를 range(30)으로 늘린다. 퀴즈는 어떻게 이어지나?

```
cd examples/vol2/14-llm-lora && uv run python b03_next.py
```

로짓 모양이 (1, 7, 151936)일 때, 다음 토큰을 고르는 데 쓰는 것은?

Q

가 로짓[0, 0]: 첫 자리의 점수

나 로짓[0, -1]: 마지막 자리의 점수

다 7자리 점수의 평균

라 151936개 점수 중 가장 작은 것

16차시 정리

from_pretrained(이름) 한 줄로 모델과 토크나이저를 받는다. 5억 개짜리 모델도 **nn.Module**

토크나이저: 글 → 번호 → 글. 같은 뜻이어도 한국어는 토큰이 더 많다

LLM이 하는 일은 **다음 토큰의 점수표** 하나. 고르고, 붙이고, 다시 넣으면 글이 된다

다음 차시: 1등만 고르지 않는 법(온도, top-k, top-p)과 채팅 템플릿

17차시

17

디코딩, 프롬프트와 채팅 템플릿

책 14.3 ~ 14.4절

학습 목표

- 탐욕·샘플링·온도·top-k·top-p가 다음 토큰 고르기를 어떻게 바꾸는지 설명한다
- 채팅 템플릿과 시스템 프롬프트로 모델에게 일을 시킨다
- 예시(few-shot)와 대화 기록을 메시지 리스트로 다룬다

14.3 온도: 점수를 나누고 확률로

```
입력 = 토크나이저("점심 메뉴로 추천하는 음식은", return_tensors="pt")
with torch.no_grad():
    로짓 = 모델(**입력).logits[0, -1]

후보 = 로짓.topk(5).indices  # 원래 점수 기준 상위 5개 후보
print("후보:", [토크나이저.decode([n]) for n in 후보.tolist()])
for 온도 in [0.5, 1.0, 2.0]:
    확률 = torch.softmax(로짓 / 온도, dim=-1)  # 점수를 온도로 나눈 뒤 확률로
    print(f"온도 {온도}: 상위 5개 {[round(p, 3) for p in 확률[후보].tolist()]}"
          f"  나머지 합 {1 - 확률[후보].sum().item():.3f}")
```

```
후보: [' 무엇', '?', ' 다음과', " '", ' 어떤']
온도 0.5: 상위 5개 [0.967, 0.016, 0.004, 0.002, 0.001]  나머지 합 0.010
온도 1.0: 상위 5개 [0.414, 0.054, 0.027, 0.016, 0.016]  나머지 합 0.473
온도 2.0: 상위 5개 [0.014, 0.005, 0.004, 0.003, 0.003]  나머지 합 0.971
```

14.3 온도를 낮추면 날카롭게, 높이면 평평하게



진한 청록 = 1등 ' 무엇' · 옅은 청록 = 2~5등 합 · 회색 = 나머지 15만여 개의 합 (막대 전체 = 확률 1)

온도	1등 ' 무엇'	나머지 합	뽑으면
0.5	**0.967**	0.010	거의 탐욕처럼 늘 같은 답
1.0 (원래)	0.414	0.473	모델이 낸 확률 그대로
2.0	0.014	**0.971**	엉뚱한 토큰이 자주 섞인다

17차시 · 14.3 온도 · 예제 14-llm-lora/b05_temperature.py 출력을 막대로 (2~5등 합 = 출력의 둘째~다섯째 값을 더한 것)

14.3 top-k와 top-p: 엉뚱한 후보는 아예 빼기

```
def top_k_거르기(확률: torch.Tensor, k: int) -> torch.Tensor:
    남길 = torch.zeros_like(확률)
    값, 번호 = 확률.topk(k)
    남길[번호] = 값
    return 남길 / 남길.sum()
def top_p_거르기(확률: torch.Tensor, p: float) -> torch.Tensor:
    정렬값, 정렬번호 = 확률.sort(descending=True)
    누적 = 정렬값.cumsum(dim=0)
    버릴 = (누적 - 정렬값) >= p  # 앞까지의 합이 이미 p 이상이면 버린다
    정렬값[버릴] = 0.0
    남길 = torch.zeros_like(확률).scatter(0, 정렬번호, 정렬값)
    return 남길 / 남길.sum()
```

```
top-k=3 : [0.61, 0.244, 0.146, 0.0, 0.0, 0.0]
top-p=0.85: [0.556, 0.222, 0.133, 0.089, 0.0, 0.0]
```

- **top-k**: 높은 k개만
- **top-p**: 누적 확률이 p에 닿을 때까지

14.3 1000번 뽑아 보면: 우주선이 55번

후보	김밥	라면	비빔밥	떡볶이	냉면	우주선
원래 확률	0.50	0.20	0.12	0.08	0.06	0.04
top-k=3	0.61	0.244	0.146	0.0	0.0	0.0
top-p=0.85	0.556	0.222	0.133	0.089	0.0	0.0
그대로 1000번	500	206	125	59	55	**55**
top-p 1000번	566	228	117	89	0	**0**

점심 메뉴를 묻는데 1000번 중 55번은 "우주선". top-p로 거르면 한 번도 나오지 않는다

top-k는 남길 개수를 고정, top-p는 분포 모양에 맡긴다. 실무에서는 top-p를 더 많이 쓴다

17차시 · 14.3 top-k와 top-p · 예제 14-llm-lora/b06_topk_topp.py 출력 네 줄을 표로 (원래 확률은 예제용 숫자)

14.3 generate로 비교: "비 오는 날 창밖을 보며"

설정 / 씨앗	이어 쓴 25토큰 (출력 그대로)
탐욕 / 1	', 그는 놀라운 일들을 보았습니다. 그는 놀라운 일들을 보았'
탐욕 / 2	', 그는 놀라운 일들을 보았습니다. 그는 놀라운 일들을 보았'
온도 0.7, top-p 0.9 / 1	', 이어질 수 없는 우주의 진화와 재생의 문화가 있습니다. 2012년'
온도 0.7, top-p 0.9 / 2	' "오늘 밤은 달빛이 매력적이고, 움직이는 생명체들의 수요를'
온도 1.5 / 1	' 목야()})\n{\\hangfmt}`,ห้องที่ 관});\n\n\n.perform_pipe("DMETHOD皋 design score efficiently prioritize detail activity cap'
온도 1.5 / 2	' 통해弼 tắt reacts bc 달 에Congress 리 Toni compens foreverمنح Tedו Failed doğ의 buz Accessed быстро לכ fullWidth'

탐욕은 씨앗과 상관없이 같고 벌써 되풀이한다 · 0.7/0.9는 다르면서도 한국어 꼴 · 1.5(거르기 끔)는 한 번 미끄러지면 계속 미끄러진다

14.4 대화는 사실 하나의 긴 글

```
대화 = [
    {"role": "system", "content": "너는 친절한 파이썬 선생님이야."},
    {"role": "user", "content": "리스트가 뭐예요?"},
]
글 = 토크나이저.apply_chat_template(대화, tokenize=False, add_generation_prompt=True)
print(글)
```

```
<|im_start|>system
너는 친절한 파이썬 선생님이야.<|im_end|>
<|im_start|>user
리스트가 뭐예요?<|im_end|>
<|im_start|>assistant
```

system: 대화 전체에 걸리는 태도와 규칙

user: 사용자가 한 말 · **assistant**: 모델이 한 말

마지막 줄 = "이제 모델이 말할 차례"

템플릿은 모델마다 다르다. 문자열을 손으로 조립하지 말고 그 모델 토크나이저의 **apply_chat_template**을 쓴다

17차시 · 14.4 대화는 사실 하나의 긴 글입니다 · 예제 14-llm-lora/b08_template.py (핵심 부분, 출력의 빈 줄·특수 토큰 줄 생략)

14.4 시스템 프롬프트: "고양이에 대해 알려 줘."

시스템 지시	답 (출력 그대로, 50토큰까지)	지켰나
답은 반드시 한 문장으로 해.	'고양이는 사람의 식사와 휴식을 즐기며, 농담과 이야기를 나누는 동물입니다.'	지킴
모든 답을 JSON으로 해. 키는 '요약' 하나뿐이야.	'{\n "요약": "고양이는 고유한 특징과 장점을 가지고 있으며, 특히 농사와 식물栽培에 유명합니다. 고양이는 다양한 종류의 식물과 동물로'	형식은 지킴
모든 답을 영어로만 해.	'고양이는 매우 유명한 동물입니다. 그들은 다양한 종류가 있으며, 특히 고유의 특징과 행동은 다음과 같습니다:\n\n1. 고양이는 빨간색 또는 검은색으로 주요 색'	무시
너는 해적이야. 해적 말투로 답해.	'고양이는 매우 건강하고 유쾌한 동물입니다. 고양이는 편안하게 웃으며, 다양한 활동을 즐길 수 있습니다. 고양이는 높은 체중과 강력한'	무시

시스템 프롬프트는 **부탁이지 명령이 아니다**. 결과를 확인하는 코드를 따로 둔다

14.4 예시를 보여 주기: few-shot

```
예시 = [
    {"role": "system", "content": "상품 후기를 읽고 '긍정' 또는 '부정' 한 단어로만 답해."},
    {"role": "user", "content": "배송이 빠르고 품질도 좋아요."},
    {"role": "assistant", "content": "긍정"},
    {"role": "user", "content": "한 번 빨았더니 색이 다 빠졌어요."},
    {"role": "assistant", "content": "부정"},
]
```

후기	예시 없이	예시 두 개
가격 대비 정말 만족합니다.	'물론입니다! "가격 대비 정말 만족합니다."는 긍정적인 표현'	'긍정'
사이즈가 너무 작아서 반품했어요.	'죄송합니다, 저는 인공지능으로서 사이트를 운영하고 있습니다. 하지만 개인'	'부정'
포장은 꼼꼼했는데 맛이 별로네요.	'물론입니다! "다음 후기는 긍정이야 부정이야?"와'	'긍정' (틀림)
생각보다 튼튼하고 예뻐요.	'물론입니다! "다음 후기는 긍정이야"라는 문장은 "'	'긍정'

가중치는 하나도 안 바뀌었는데 **형식**을 따라 한 단어로 답한다. 넷 중 셋을 맞혔다

14.4 모델은 대화를 기억하지 않는다

```
기록 = [{"role": "user", "content": "내 이름은 민준이야."}]
기록.append({"role": "assistant", "content": 묻기(기록)})
새_질문 = {"role": "user", "content": "내 이름이 뭐라고 했지?"}
print("기록 없이:", 묻기([새_질문]))
print("기록 함께:", 묻기(기록 + [새_질문]))
```

출력	내용 (출력 그대로)
첫 답	안녕하세요! 저는 AI 어시스턴트인 Qwen입니다. 어떻게 도와드릴까요?
기록 없이	저는 "Qwen"라는 이름을 가지고 있습니다. 이 이름은 "Quantum Neural Network"의 약자로, AI 분야에서 중요한 역할을 하는 인공지능
기록 함께	당신의 이름은 "민준"이라고 합니다.
메시지 수	3

챗봇의 기억 = 프로그램이 지난 대화를 **모두 이어 붙여 매번 다시 넣는 것**. 길어질수록 느려지고, 최대 길이를 넘으면 앞을 잘라야 한다

직접 실행 10분

1. b07_generate.py의 설정들에 "온도 0.3, top-p 0.9"를 더해 씨앗 1, 2로 돌린다. 온도 0.7과 무엇이 다른가?
2. b10_fewshot.py의 예시 끝에 user "디자인은 예쁜데 금방 고장 났어요." / assistant "부정" 짝을 하나 더 넣는다. "포장은 꼼꼼했는데 맛이 별로네요."를 이제 맞히나?
3. b11_history.py에서 기록을 함께 보내되 첫 메시지만 빼면 어떻게 답하는지 예상하고 돌려 본다

```
dict(끄기, do_sample=True, temperature=0.3, top_p=0.9)
```

17차시 · 직접 실행 · 파일마다 이 컴퓨터(CPU)에서 6~11초 · 샘플링 결과는 컴퓨터마다 다를 수 있다

챗봇이 앞의 대화를 기억하는 것처럼 보이는 이유는?

Q

가 대화할 때마다 모델 가중치에 내용이 저장된다

나 시스템 프롬프트가 기억 기능을 켠다

다 프로그램이 지난 대화를 모두 이어 붙여 매번 다시 넣는다

라 온도를 낮추면 기억이 오래 간다

17차시 정리

탐욕은 늘 같은 답. **온도**로 분포를 날카롭게/평평하게, **top-k·top-p**로 엉뚱한 후보를 거른 뒤 뽑는다

apply_chat_template으로 system·user·assistant 메시지를 모델의 형식에 맞춘다

예시는 대화처럼 넣는다(**few-shot**). 모델은 기억이 없으니 대화 기록을 매번 함께 보낸다

다음 차시: 작은 모델의 한계를 정직하게 보고, 메모리를 계산한다

18차시

18

작은 모델의 한계, 메모리 계산과 양자화

책 14.5 ~ 14.6절

학습 목표

- 작은 모델이 그럴듯한 거짓을 만드는 이유를 셋으로 말한다
- 모델 크기·자료형·KV 캐시로 메모리를 계산한다
- 정수 칸에 담는 양자화를 직접 흉내 내고 비트 수의 영향을 잰다

14.5 일부러 사실을 물어보면

질문	답 (출력 그대로, 45토큰까지)	판정
파이썬에서 리스트와 튜플의 차이를 한 문장으로 설명해 줘.	파이썬에서 리스트와 튜플의 차이는 다음과 같습니다: 1. 리스트는 원소가 변경될 수 있는 자료형입니다. 2. 튜플은 원소가 변경되지 않아서	내용은 맞음, "한 문장" 무시
In one sentence, what is the difference between a list and a tuple in Python?	A list and a tuple in Python are both used to store collections of items, but they differ in their structure and operations: lists are mutable (changeable) and can be indexed, while tuples are immutable (unchangeable	한 문장, 정확
37 곱하기 24는 얼마야? 숫자만 답해.	37 곡 24은 105입니다.	틀림 (888)
세종대왕이 만든 문자는?	세종대왕이 만든 문자는 "서울"입니다. 세종대왕은 19세기의 대한민국의 세종도시에 거주한 대한민국의 천년	틀림 (한글)
대한민국에서 가장 높은 산은?	대한민국에서 가장 높은 산은 울릉산입니다. 물론, 이는 대략적인 기준으로 생각해 보시면 좋을 것 같습니다: 1. 울릉	틀림 (한라산)

그럴듯한 말투로 사실이 아닌 것을 지어내는 것: **환각(hallucination)**

14.5 왜 지어낼까: 세 가지 이유

모델이 작다

사실을 담을 숫자가 5억 개 남짓. 큰 모델은 수천억 개

문장 잇는 능력은 남아도 정확한 사실은 자주 빠진다

한국어를 적게 봤다

주로 영어·중국어로 학습

답에 한자가 섞이고, 같은 질문도 영어 답이 낫고, 토큰도 잘게 쪼개진다

그럴듯함을 배운다

목표는 처음부터 끝까지 "다음에 올 법한 토큰"

곱셈을 계산하지 않고, 사실을 찾아보지 않는다

형식 맞추기, 짧게 정리하기는 잘한다. 큰 모델로 바꾸면 답은 좋아지지만 **메모리**가 들고, **라이선스**도 크기마다 다시 확인한다

14.6 내 노트북에 들어갈까: 칸 크기 × 칸 개수

```
GB = 1024 ** 3
# 4장 4.3절: 메모리 = 칸 크기(element_size) × 칸 개수(numel)
바이트 = sum(p.element_size() * p.numel() for p in 모델.parameters())
print(f"float32로 올린 이 모델: {바이트 / GB:.2f} GB")
print("입력 임베딩과 출력층이 같은 가중치인가:",
      모델.get_input_embeddings().weight is 모델.get_output_embeddings().weight)

# 같은 계산을 크기별·자료형별로: 칸당 바이트 수만 바꾸면 된다
for 크기, 개수 in [("0.5B", 494_032_768), ("7B", 7_000_000_000), ("70B", 70_000_000_000)]:
    칸 = {"float32": 4, "bfloat16": 2, "int8": 1, "int4": 0.5}
    print(f"{크기:>4}: " + ", ".join(f"{k} {개수 * v / GB:6.1f}GB" for k, v in 칸.items()))
```

```
float32로 올린 이 모델: 1.84 GB
입력 임베딩과 출력층이 같은 가중치인가: True
```

4억 9천만 × 4바이트 ÷ 1024^3 = 1.84GB. 어휘 15만 개짜리 행렬(1억 3천만 개 넘음)을 입력·출력이 **함께 써서** 한 번만 둔다

18차시 · 14.6 메모리 계산과 양자화 맛보기 · 예제 14-llm-lora/b13_memory.py (핵심 부분, 크기별 표는 다음 장)

14.6 크기와 자료형으로 본 가중치 메모리

모델 크기	float32 (4)	bfloat16 (2)	int8 (1)	int4 (0.5)
0.5B (오늘 모델)	1.8GB	0.9GB	0.5GB	0.2GB
7B	26.1GB	13.0GB	6.5GB	**3.3GB**
70B	260.8GB	130.4GB	65.2GB	**32.6GB**

큰 모델을 노트북에서 돌린다 = 대부분 **칸을 줄여서** 담는다는 뜻

실측(이 컴퓨터, float32 CPU): 불러와 쓰기 약 3.3GB, LoRA 학습 약 7.9GB

14.6 생성할 때 따로 쌓이는 메모리: KV 캐시

```
설정 = AutoConfig.from_pretrained(이름)
헤드_크기 = 설정.hidden_size // 설정.num_attention_heads
토큰당 = 2 * 설정.num_hidden_layers * 설정.num_key_value_heads * 헤드_크기 * 2  # bfloat16 기준
```

```
키·값 헤드 2개, 헤드 크기 64, 토큰당 12,288바이트
토큰 32,768개를 기억하면 KV 캐시 0.38 GB
키·값 헤드가 쿼리 헤드처럼 14개였다면 2.62 GB
```

2(키와 값) × 24층 × 2헤드 × 64 × 2바이트 = **12,288바이트**

쿼리 헤드 7개가 키·값 헤드 하나를 함께 쓴다(GQA): 7배 절약

14.6 bfloat16으로 절반만 쓰기

```
for 자료형 in [torch.float32, torch.bfloat16]:
    모델 = AutoModelForCausalLM.from_pretrained(이름, dtype=자료형).eval()
    with torch.no_grad():
        로짓 = 모델(**입력).logits[0, -1].float()
        생성 = 모델.generate(**입력, max_new_tokens=20, do_sample=False,
                             repetition_penalty=1.1)
    print(f"{str(자료형):15} {모델.get_memory_footprint() / 1024**3:.2f} GB  " ...)
```

```
torch.float32   1.84 GB  '1 더하기 1은 2입니다.\n\n더하기 연산자는 두 수를 더합니다'
torch.bfloat16  0.92 GB  '1 더하기 1은 2입니다.\n\n더하기 연산자는 두 수를 더합니다'
다음 토큰 점수 차이: 평균 0.089, 최대 0.458
1등 토큰이 같은가: True
```

메모리는 정확히 절반, 문장은 똑같다. 딥러닝은 끝자리의 작은 오차에 둔감해서 요즘 LLM은 대부분 bfloat16으로 배포된다

14.6 양자화: 자에 눈금을 새기듯 정수 칸에 담기

```
def 양자화_왕복(w: torch.Tensor, 비트: int) -> torch.Tensor:
    최대정수 = 2 ** (비트 - 1) - 1  # int8이면 127, int4면 7
    배율 = w.abs().amax(dim=1, keepdim=True) / 최대정수  # 행마다 하나 (keepdim: 4장 4.6절)
    정수 = (w / 배율).round().clamp(-최대정수, 최대정수)  # 이 정수만 저장하면 된다
    return 정수 * 배율

w = torch.tensor([[0.12, -0.50, 0.33, 0.07]])
```

```
원래   : tensor([[ 0.1200, -0.5000,  0.3300,  0.0700]])
int8   : tensor([[ 0.1181, -0.5000,  0.3307,  0.0709]])
int4   : tensor([[ 0.1429, -0.5000,  0.3571,  0.0714]])
```

- int8 눈금 간격 0.5/127: 거의 그대로
- int4 눈금은 -7~7 열다섯 개: 0.12가 0.1429로

14.6 모델 전체에 적용하면: int8은 그대로, int4는 망가진다

```
for 층 in 모델.model.layers.modules():  # 트랜스포머 층 안의 선형층만
    if isinstance(층, torch.nn.Linear):
        층.weight.copy_(양자화_왕복(층.weight, 비트))
```



막대 = 다음 토큰 정답 확률의 기하평균 · 손실은 2.189 → 2.200 → 3.968

int4가 망가진 건 행마다 배율을 **하나만** 썼기 때문. 실제 4비트 도구는 작은 묶음마다 배율을 둔다

18차시 · 14.6 양자화 맛보기 · 예제 14-llm-lora/b15_quant.py (핵심 부분, 출력 마지막 세 줄을 막대로)

직접 실행 10분

1. b15_quant.py의 비트 목록을 [None, 8, 6, 5, 4, 3]으로 바꿔 손실을 잰다. 어디서부터 급격히 나빠지나?
2. 토큰당 12,288바이트일 때 4,096토큰 대화 8개를 동시에 처리하면 KV 캐시는 몇 GB? 손으로 계산하고 코드로 확인한다
3. b12_limits.py 질문 목록에 "What is the highest mountain in South Korea?"를 더한다. 영어로 물으면 맞히나?

```
print(f"{12_288 * 4_096 * 8 / 1024**3:.2f} GB")
```

7B 모델을 bfloat16으로 올리면 가중치만 약 몇 GB 일까요?

Q

가 3.3GB

나 6.5GB

다 13.0GB

라 26.1GB

18차시 정리

작은 모델은 그럴듯한 거짓(**환각**)을 만든다: 작은 크기, 적은 한국어, "그럴듯함"이 목표

메모리 = **파라미터 수 × 칸 크기**. 생성할 때는 KV 캐시(2 × 층 × 키·값 헤드 × 헤드 크기 × 칸 크기 × 토큰 수)가 더해진다

bfloat16은 절반에 거의 같은 답. 양자화는 정수 칸과 배율, 비트를 너무 줄이면 망가진다

다음 차시: 모델 전체가 아니라 아주 작은 부분만 학습하는 LoRA

19차시

19

LoRA: 고칠 부분만 작게

책 14.7 ~ 14.8절

학습 목표

- 작은 행렬 두 개의 곱이 왜 "랭크가 낮은 변화"인지 설명한다
- LoRA 층을 nn.Module로 직접 만들고 장난감 문제로 확인한다
- peft로 LLM에 LoRA를 붙이고 학습할 숫자를 손으로 센다

14.7 교과서를 다시 쓰지 말고 포스트잇을 붙이기



B: 896 × r (0에서 시작)

A: r × 896 (작은 무작위 값)

고칠 양 **ΔW = B @ A**만 학습

통째로 고치면 Adam이 숫자마다 둘을 더 든다. 가중치 1.84GB에 약 5.5GB가 더, 7B면 100GB가 넘는다

LoRA(Low-Rank Adaptation): W는 그대로 두고 작은 행렬 두 개만 붙여 고친다

14.7 작은 행렬 두 개로 큰 행렬 만들기: 랭크

```
a = torch.tensor([[1.0], [2.0], [3.0]])  # (3, 1)
b = torch.tensor([[10.0, 20.0, 30.0, 40.0]])  # (1, 4)
표 = a @ b  # (3, 4): 숫자 7개로 12개짜리 표를 만든다
print("랭크:", torch.linalg.matrix_rank(표).item())
B, A = torch.randn(896, 8), torch.randn(8, 896)
print("(896, 8) @ (8, 896) 의 랭크:", torch.linalg.matrix_rank(B @ A).item())
```

```
tensor([[ 10.,  20.,  30.,  40.],
        [ 20.,  40.,  60.,  80.],
        [ 30.,  60.,  90., 120.]])
랭크: 1
(896, 8) @ (8, 896) 의 랭크: 8
```

896 × 896	학습할 숫자	비율
통째로	802,816	100%
랭크 1	1,792	0.2%
랭크 4	7,168	0.9%
랭크 8	14,336	1.8%
랭크 16	28,672	3.6%
랭크 64	114,688	14.3%

14.7 LoRA 층을 손으로 만들기

```
class LoRA선형층(nn.Module):
    def __init__(self, 원래: nn.Linear, r: int = 4, alpha: int = 8) -> None:
        super().__init__()
        self.원래 = 원래
        self.원래.requires_grad_(False)  # 원래 가중치는 얼린다 (6장 6.4절)
        self.A = nn.Parameter(torch.randn(r, 원래.in_features) * 0.01)  # 작은 무작위 값
        self.B = nn.Parameter(torch.zeros(원래.out_features, r))  # 0에서 시작
        self.배율 = alpha / r

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.원래(x) + (x @ self.A.T @ self.B.T) * self.배율

원래 = nn.Linear(64, 64)
층 = LoRA선형층(원래, r=4)
```

원래 층의 출력에 **x @ A.T @ B.T**(랭크 r의 좁은 통로로 줄였다가 다시 펴는 길)를 더한다. 기초 과정 nn.Linear의 x @ weight.T와 같은 모양

14.7 눈여겨볼 세 군데

```
처음에는 원래 층과 출력이 같은가: True
A               (4, 64)    학습함=True
B               (64, 4)    학습함=True
원래.weight      (64, 64)   학습함=False
원래.bias        (64,)      학습함=False
학습하는 숫자 512개 / 전체 4,672개
```

B는 0에서

처음엔 B @ A = 0. 원래 실력에서 출발한다 (True)

A는 작은 무작위

둘 다 0이면 서로의 기울기가 영원히 0

배율 alpha / r

랭크를 바꿔도 고치는 세기가 비슷하게. 보통 alpha = r의 1~2배

14.7 정말 낮은 랭크로 배워질까: 정답을 아는 문제

```
진짜_변화 = torch.randn(64, 2) @ torch.randn(2, 64) * 0.1
for r in [1, 2, 4]:
    층 = LoRA선형층(원래, r=r, alpha=r)  # 배율 1로 맞춰 비교
    옵티마이저 = torch.optim.Adam([층.A, 층.B], lr=0.01)
    for 단계 in range(1, 1501):
        손실 = nn.functional.mse_loss(층(x), y)
        옵티마이저.zero_grad()
        손실.backward()
        옵티마이저.step()
    배운_변화 = 층.배율 * 층.B @ 층.A
```

```
r=1: 손실 0.502039, 배운 변화와 진짜 변화의 상대 오차 0.658
r=2: 손실 0.000000, 배운 변화와 진짜 변화의 상대 오차 0.000
r=4: 손실 0.000000, 배운 변화와 진짜 변화의 상대 오차 0.000
합친 층과 LoRA 층의 출력이 같은가: True
```

r=1은 통로가 좁아 못 배우고, r≥2는 정확히 배웠다. 다 배우면 $W + \text{배율} \times (B @ A)$로 보통 층 하나에 합쳐 추가 계산 없이 쓴다

19차시 · 14.7 정말 낮은 랭크로 배워질까? · 예제 14-llm-lora/b18_lora_toy.py (핵심 부분)

14.8 peft로 LLM에 LoRA 붙이기

```
from peft import LoraConfig, get_peft_model

설정 = LoraConfig(
    r=8,  # 작은 행렬의 랭크
    lora_alpha=16,  # 배율 = lora_alpha / r = 2
    lora_dropout=0.05,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],  # 어텐션의 선형층 네 개
    task_type="CAUSAL_LM",
)
모델 = get_peft_model(모델, 설정)
모델.print_trainable_parameters()
```

```
trainable params: 1,081,344 || all params: 495,114,112 || trainable%: 0.2184
```

5억 개 중 **108만 개**(0.22%)만 고친다. 고를 층은 이름으로: 7~8차시에서 만든 쿼리·키·값·출력 층

14.8 바뀐 층을 들여다보고 손으로 세기

```
lora.Linear(
  (base_layer): Linear(in_features=896, out_features=128, bias=True)
  (lora_dropout): ModuleDict(
    (default): Dropout(p=0.05, inplace=False)
  )
  (lora_A): ModuleDict(
    (default): Linear(in_features=896, out_features=8, bias=False)
  )
  (lora_B): ModuleDict(
    (default): Linear(in_features=8, out_features=128, bias=False)
  )
  (lora_embedding_A): ParameterDict()
  (lora_embedding_B): ParameterDict()
  (lora_magnitude_vector): ModuleDict()
)
손으로 센 LoRA 숫자: 1,081,344개
붙이기 전후 출력이 같은가: True
```

k_proj = base_layer 옆에 lora_A(896 → 8), lora_B(8 → 128). 손으로 만든 구조 그대로

층마다 r × (입력 + 출력): 블록 하나 45,056개 × 24블록 = **1,081,344**

출력 128 = 키·값 헤드 2개 × 64 (18차시 GQA)

직접 실행 10분

1. b18_lora_toy.py의 진짜 변화를 랭크 3으로, 랭크 목록을 [1, 2, 3, 4]로 바꾼다. 어느 랭크부터 손실이 0이 되나?
2. b19_peft.py의 target_modules에 "gate_proj", "up_proj", "down_proj"를 더한다. 학습하는 숫자는 몇 개, 몇 %?
3. 같은 파일에서 r=16, lora_alpha=32로 바꾸면 학습하는 숫자는?

```
진짜_변화 = torch.randn(64, 3) @ torch.randn(3, 64) * 0.1
```

LoRA에서 B를 0으로 시작하는 까닭은?

Q

가 학습을 더 빨리 끝내려고

나 처음에는 원래 모델과 출력이 똑같게 하려고

다 저장할 어댑터 파일을 작게 하려고

라 A의 기울기를 0으로 만들려고

19차시 정리

LoRA: 원래 W는 얼리고 ΔW = B @ A(랭크 r)만 학습. B는 0에서, 배율 alpha / r

필요한 변화의 랭크만큼이면 정확히 배운다. 다 배우면 **W에 합쳐** 추가 계산 없이 쓴다

peft: LoraConfig(r, lora_alpha, target_modules) → get_peft_model → 0.22%만 학습

다음 차시: 1권 도우미로 미세조정하고, 무엇을 배우고 무엇을 못 배우는지 잰다

19차시 · 정리 · 복습: 책 14.7 ~ 14.8절

20차시

20

실습: 1권 도우미로 미세조정하기

책 14.9 ~ 14.10절

학습 목표

- 질문 부분을 -100으로 가린 지시-응답 데이터를 만든다
- LoRA로 작은 LLM을 미세조정하고 어댑터만 저장한다
- 무엇을 배우고 무엇을 못 배웠는지 지표를 나눠 잰다

14.9 목표와 데이터: 짧게 답하고 출처를 붙이는 도우미

학습 문답 한 쌍 (일권문답.py)

Q. 변수란 무엇인가요?

A. 값에 붙이는 이름표예요. `나이 = 20`처럼 등호로 이름과 값을 묶어 두고 이름으로 다시 꺼내 씁니다. (참고: 1권 2장)

42쌍

학습 문답. 답은 모두 "(참고: 1권 N 장)"으로 끝난다

4쌍

시험 문답: 학습 내용을 다르게 묻기

2쌍

시험 문답: 학습에 없는 주제(f-string, 정규 표현식)

14.9 질문은 가리고 답만 배우기: -100

```
def 예제_만들기(토크나이저, 질문: str, 답: str) -> tuple[torch.Tensor, torch.Tensor]:
    """입력 번호와 정답 번호를 만든다. 질문 부분의 정답은 -100으로 가려 손실에서 뺀다."""
    앞 = 토크나이저(프롬프트(토크나이저, 질문), add_special_tokens=False)["input_ids"]
    뒤 = 토크나이저(답 + "<|im_end|>", add_special_tokens=False)["input_ids"]
    return torch.tensor(앞 + 뒤), torch.tensor([-100] * len(앞) + 뒤)
```

앞: 시스템 지시 + 질문 (채팅 템플릿)

정답 = -100 -100 -100 ...

뒤: 답 + <|im_end|>

정답 = 답의 번호

- 교차 엔트로피는 정답이 -100인 자리를 빼고 계산한다(ignore_index=-100이 기본값)
- 질문을 따라 쓰는 법이 아니라 **답을 쓰는 법만** 배우고, 끝의 <|im_end|>로 **멈추는 법**까지 배운다

14.9 미세조정 전: 기준선부터 잰다

시험 질문	모델 답 (첫 줄, 출력 그대로)
튜플은 값을 바꿀 수 있나요?	네, 템플을 사용하여 값을 변경할 수 있습니다. 예를 들어, 다음과 같은 코드를 작성하면 `a`와 `b`의 값이 서로 바뀝니다:
with 문으로 파일을 열면 무엇이 좋은가요?	with 문은 파일을 항상 닫아주어야 합니다. 이로 인해 파일이 닫히지 않으면 오류를 발생시킵니다. 따라서 파일을 닫아주는 코드를 작성해야 합니다.
yield는 무엇을 하나요?	yield는 Python에서 함수를 호출하거나 반복할 때 사용하는 특별한 기능입니다.

```
[미세조정 전] 학습 문답: 답 손실 2.782, 장 번호 자리 손실 2.179, 장 번호 맞힘 10%
[미세조정 전] 시험 문답: 답 손실 2.974, 장 번호 자리 손실 2.024, 장 번호 맞힘 0%
```

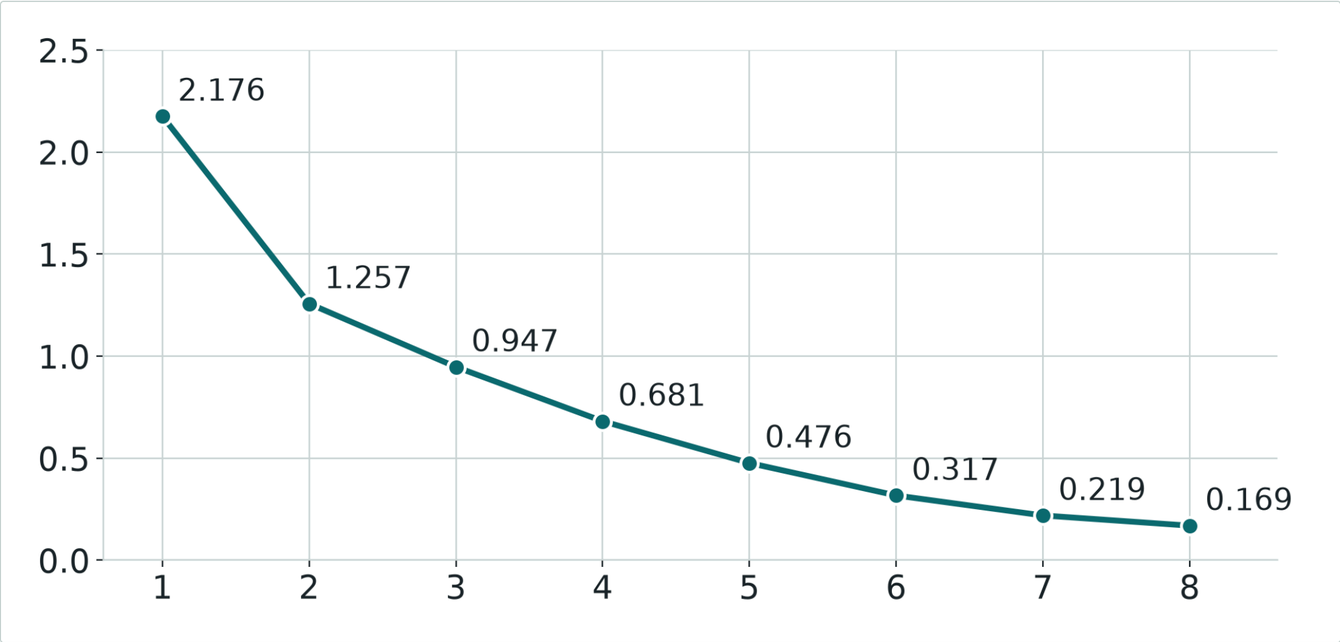
20차시 · 14.9 미세조정 전 · 예제 14-llm-lora/b20_before.py 출력 (첫 줄 "학습 문답 42개, 시험 문답 6개" 생략)

14.9 LoRA로 학습하기: 기초 과정의 다섯 줄 그대로

```
모델 = get_peft_model(모델, 설정)  # r=8, lora_alpha=16, 어텐션 네 층 (19차시와 같음)
예제들 = [예제_만들기(토크나이저, 질문, 답) for 질문, 답 in 학습_문답]
옵티마이저 = torch.optim.AdamW([p for p in 모델.parameters() if p.requires_grad], lr=1e-3)
배치_크기 = 8
for 에폭 in range(1, 9):
    모델.train()
    순서 = torch.randperm(len(예제들), generator=생성기).tolist()
    for i in range(0, len(순서), 배치_크기):
        묶음 = [예제들[j] for j in 순서[i:i + 배치_크기]]
        입력 = pad_sequence([x for x, _ in 묶음], batch_first=True, padding_value=토크나이저.pad_token_id)
        정답 = pad_sequence([y for _, y in 묶음], batch_first=True, padding_value=-100)
        마스크 = pad_sequence([torch.ones(len(x), dtype=torch.long) for x, _ in 묶음], batch_first=True)
        손실 = 모델(input_ids=입력, attention_mask=마스크, labels=정답).loss
        옵티마이저.zero_grad()
        손실.backward()
        옵티마이저.step()
모델.save_pretrained(어댑터_경로)  # LoRA 부분(A, B)만 저장된다
```

채운 자리는 정답 -100, 마스크 0: 손실에도 어텐션에도 끼어들지 않는다. 42개를 8개씩, 8에폭 = 48번 고친다

14.9 학습 손실 2.176 → 0.169, 저장은 4.15MB



가로 에폭 · 세로 학습 손실 (예제 42개 평균)

```
README.md: 0.00 MB
adapter_config.json: 0.00 MB
adapter_model.safetensors: 4.15 MB
```

A와 B만 담은 **어댑터**. 원래 모델 1.84GB의 0.2%쯤. 일마다 다른 어댑터를 바꿔 끼운다

14.9 미세조정 전후: 평균 손실과 장 번호를 나눠 보면

지표	학습 전	학습 후	시험 전	시험 후
답 손실	2.782	**0.123**	2.974	**1.976**
장 번호 자리 손실	2.179	1.862	2.024	1.906
장 번호 맞힘	10%	10%	0%	17%

답 전체 손실은 **20분의 1**보다 작아졌다

장 번호 자리는 **거의 그대로**. 학습 문답에서도 10%

평균 손실만 봤다면 "다 배웠다"고 착각했을 것. 숫자 하나로 판단하지 말고 **무엇을 틀리는지** 본다

20차시 · 14.9 미세조정 후 · 예제 14-llm-lora/b20_before.py, b22_after.py 출력의 지표 줄을 표로

14.9 미세조정 후의 답: 말투는 완벽, 사실은 그대로

질문	기준 (데이터의 답)	모델 (출력 그대로)
변수란 무엇인가요? (학습)	값에 붙이는 이름표예요. `나이 = 20`처럼 등호로 이름과 값을 묶어 두고 이름으로 다시 꺼내 씁니다. (참고: 1권 2장)	값에 붙이는 이름표예요. `나이 = 20`처럼 등호로 이름과 값을 묶어 두고 이름으로 다시 꺼내 씁니다. (참고: 1권 6장)
튜플은 값을 바꿀 수 있나요? (시험)	아니요. 튜플은 한번 만들면 바꿀 수 없습니다. 바꿔야 하면 리스트를 씁니다. (참고: 1권 6장)	네, 값을 바꿀 수 있습니다. `tup`를 객체 인스턴스로 만들어 주면 값을 바꿀 수 있어요. (참고: 1권 9장)
부모 클래스의 메서드를 자식 클래스에서 부르려면 어떻게 하나요? (시험)	`super().메서드이름()`처럼 super()를 거쳐 부릅니다. (참고: 1권 13장)	`__class__` 속성을 물어 둔 다음 `__class__`로 부른 자식을 다시 물어 둔 방법입니다. (참고: 1권 9장)
f-string은 무엇인가요? (새 주제)	문자열 앞에 f를 붙이고 중괄호 안에 변수를 넣어 값을 끼워 넣는 문법입니다. (참고: 1권 2장)	`f-string`로 씁니다면 자동으로 f와 {}를 바꾸고, 값을 꼭 넣어야 해요. (참고: 1권 6장)

모두 한두 문장으로 끝나고 **(참고: 1권 N장)**을 붙였다. 그런데 장 번호와 내용은 틀린다

14.9 왜 형식은 배우고 사실은 못 배웠나

답 문장의 대부분

바로 앞 몇 토큰만 보면 이어 쓸 수 있다

"(참고: 1권"은 42개 답에 똑같이 나와 에폭마다 42번씩 배운다

장 번호

질문 내용에 따라 정해지는 **임의의 사실**

"변수"와 "2"를 잇는 규칙은 없고, 데이터에 한 번씩만 나온다. 8에폭이면 여덟 번

적은 데이터의 LoRA는 **"어떻게 말할지"**(말투, 형식, 길이, 규칙)에 강하고 **"무엇을 알지"**(새로운 사실)에 약하다

사실이 중요하면: 더 많은 데이터·더 큰 모델, 또는 관련 문서를 찾아 프롬프트에 넣는 RAG(3권)

직접 실행 10분

1. b22_after.py의 질문에 "딕셔너리의 get 메서드는 무엇을 하나요?"와 "오늘 점심 메뉴를 추천해 줘."를 넣어 본다. 1권과 상관없는 질문에는 어떻게 답하나?
2. 시간이 남으면: b21_train.py를 range(1, 5)(4에폭)로 바꾸고 어댑터 경로를 따로 정해 학습한 뒤 평가한다. 8에폭과 무엇이 다른가?

```
cd examples/vol2/14-llm-lora && uv run python b22_after.py
```

20차시 · 직접 실행 · 이 컴퓨터 CPU: 1번 약 11초, 2번 학습 약 54초 + 평가 약 21초 (uv 시작 포함)

사내 규정 1,000개를 정확히 답하는 챗봇이 필요하다. 가장 알맞은 방법은?

Q

가 규정 문답 몇십 개로 작은 모델에 LoRA

나 관련 규정을 찾아 프롬프트에 넣어 답하게 하기

다 온도를 낮춰 환각을 없애기

라 시스템 프롬프트에 "정확히 답해"라고 쓰기

14.10 14장 정리

주제	기억할 것
불러오기	AutoTokenizer, AutoModelForCausalLM.from_pretrained(이름, dtype=...). 모델은 nn.Module. 라이선스 확인
다음 토큰	로짓 (배치, 토큰 수, 어휘 크기). 고르고, 붙이고, 다시 넣기. generate는 KV 캐시로 빠르게
디코딩	탐욕은 늘 같은 답. 온도, top-k·top-p. 장치가 다르면 같은 씨앗도 다른 문장
프롬프트	apply_chat_template, few-shot은 대화처럼, 대화 기록은 매번 함께
한계	환각: 작은 크기, 적은 한국어, "그럴듯함"이 목표
메모리	파라미터 × 칸 크기 + KV 캐시. bfloat16은 절반, 양자화는 정수 칸과 배율
LoRA·peft	W는 얼리고 B @ A만. LoraConfig, get_peft_model, save_pretrained(어댑터만), merge_and_unload
교훈	질문은 -100으로 가린다. 말투·형식에 강하고 사실 주입에 약하다. 중요한 자리를 따로 잰다

4일차에 배운 것

1. 사전학습 모델을 불러오면 하는 일은 **다음 토큰의 점수표** 하나
2. 온도·top-k·top-p로 고르고, **채팅 템플릿**으로 일을 시킨다
3. 작은 모델은 지어낸다. 메모리는 **파라미터 × 칸 크기** + KV 캐시
4. **LoRA**: W는 얼리고 B @ A만, 0.22%를 학습한다
5. 미세조정은 **말투·형식**에 강하고 사실에 약했다

내일 5일차

- 오토인코더와 잠재 공간
- 잡음 제거, 이상 탐지
- VAE와 GAN
- 확산 모델과 조건부 확산 실습

4일차 마무리 · 복습: 책 14장 전체